



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University), Pune

Name: Devansh Singh
PRN: 25070122072
Batch: A-2
Division: A

Subject: Programming with Java

ASSIGNMENT No : 30

TITLE : Write a JSP application to demonstrate CRUD operations.

Problem Statement

Write a JSP application to demonstrate CRUD operations.

Learning Objectives

To implement Create, Read, Update, and Delete operations using JSP and a database.

Theory

CRUD stands for Create, Read, Update, and Delete, which represent the four basic database operations performed in web applications.

JSP works together with Servlets and JDBC to implement these operations. Users interact with HTML forms created using JSP, while JDBC executes SQL statements to manipulate database records.

INSERT adds new records, SELECT retrieves information, UPDATE modifies existing data, and DELETE removes unwanted records. Proper validation, exception handling, and session management improve application reliability and security.

JSP CRUD applications are widely used in employee management systems, online shopping portals, hospital management systems, banking applications, and other enterprise software.

Output Screenshots

JSP CRUD Application

CRUD stands for:

- Create - Add records
- Read - View records
- Update - Modify records
- Delete - Remove records

[Employee Management](#)

[Patient Management](#)

CODE

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4     <title>JSP CRUD</title>
5 </head>
6 <body>
7
8 <h2>JSP CRUD Application</h2>
9
10 <p>CRUD stands for:</p>
11
12 <ul>
13     <li>Create - Add records</li>
14     <li>Read - View records</li>
15     <li>Update - Modify records</li>
16     <li>Delete - Remove records</li>
17 </ul>
18
19 <a href="Exp1.jsp">Employee Management</a>
20 <br><br>
21 <a href="Exp2.jsp">Patient Management</a>
22
23 </body>
24 </html>
```

EXERCISE 1

```
1 <%@ page import="java.sql.*" %>
2 <%@ page import="com.devansh.DatabaseConnection" %>
3
4 <!DOCTYPE html>
5 <html>
6 <head>
7     <title>Employee CRUD</title>
8 </head>
9 <body>
10
11 <h2>Employee Management</h2>
12
13 <form method="post">
14
15     Employee ID:
16     <input type="number" name="emp_id" required>
17     <br><br>
18
19     Name:
20     <input type="text" name="name">
21     <br><br>
22
23     Department:
24     <input type="text" name="department">
25     <br><br>
26
27     Salary:
28     <input type="number" name="salary">
29     <br><br>
30
31     <input type="submit" name="action" value="Add">
32     <input type="submit" name="action" value="Update">
33     <input type="submit" name="action" value="Delete">
34     <input type="submit" name="action" value="View">
35
36 </form>
37
38 <%
39     String action = request.getParameter("action");
40
41     if (action != null) {
42         try {
43             Connection con = DatabaseConnection.getConnection();
```

```

44
45     int id = Integer.parseInt(request.getParameter("emp_id"));
46
47     if (action.equals("Add")) {
48
49         PreparedStatement ps = con.prepareStatement(
50             "INSERT INTO employee VALUES (?, ?, ?, ?)"
51         );
52
53         ps.setInt(1, id);
54         ps.setString(2, request.getParameter("name"));
55         ps.setString(3, request.getParameter("department"));
56         ps.setDouble(4,
Double.parseDouble(request.getParameter("salary")));
57
58         ps.executeUpdate();
59
60         out.println("<p>Employee added successfully.</p>");
61
62     } else if (action.equals("Update")) {
63
64         PreparedStatement ps = con.prepareStatement(
65             "UPDATE employee SET name=?, department=?, salary=?
WHERE emp_id=?"
66         );
67
68         ps.setString(1, request.getParameter("name"));
69         ps.setString(2, request.getParameter("department"));
70         ps.setDouble(3,
Double.parseDouble(request.getParameter("salary")));
71         ps.setInt(4, id);
72
73         ps.executeUpdate();
74
75         out.println("<p>Employee updated successfully.</p>");
76
77     } else if (action.equals("Delete")) {
78
79         PreparedStatement ps = con.prepareStatement(
80             "DELETE FROM employee WHERE emp_id=?"
81         );
82
83         ps.setInt(1, id);
84
85         ps.executeUpdate();

```

```
86
87         out.println("<p>Employee deleted successfully.</p>");
88
89     } else if (action.equals("View")) {
90
91         Statement stmt = con.createStatement();
92
93         ResultSet rs = stmt.executeQuery(
94             "SELECT * FROM employee"
95         );
96
97         out.println("<h3>Employee Records</h3>");
98
99         while (rs.next()) {
100             out.println(
101                 rs.getInt("emp_id") + " | " +
102                 rs.getString("name") + " | " +
103                 rs.getString("department") + " | " +
104                 rs.getDouble("salary") + "<br>"
105             );
106         }
107     }
108
109     con.close();
110
111 } catch (Exception e) {
112     out.println("<p>Error: " + e.getMessage() + "</p>");
113 }
114 }
115 %>
116
117 </body>
118 </html>
```

EXERCISE 2

```
1 <%@ page import="java.sql.*" %>
2 <%@ page import="com.devansh.DatabaseConnection" %>
3
4 <!DOCTYPE html>
5 <html>
6 <head>
7     <title>Patient CRUD</title>
8 </head>
9 <body>
10
11 <h2>Patient Management</h2>
12
13 <form method="post">
14
15     Patient ID:
16     <input type="number" name="patient_id" required>
17     <br><br>
18
19     Name:
20     <input type="text" name="name">
21     <br><br>
22
23     Age:
24     <input type="number" name="age">
25     <br><br>
26
27     Disease:
28     <input type="text" name="disease">
29     <br><br>
30
31     Treatment Status:
32     <input type="text" name="treatment_status">
33     <br><br>
34
35     <input type="submit" name="action" value="Add">
36     <input type="submit" name="action" value="Update">
37     <input type="submit" name="action" value="Delete">
38     <input type="submit" name="action" value="View">
39
40 </form>
41
42 <%
43     String action = request.getParameter("action");
```

```

44
45     if (action != null) {
46         try {
47             Connection con = DatabaseConnection.getConnection();
48
49             int id = Integer.parseInt(request.getParameter("patient_id"));
50
51             if (action.equals("Add")) {
52
53                 PreparedStatement ps = con.prepareStatement(
54                     "INSERT INTO patient VALUES (?, ?, ?, ?, ?)"
55                 );
56
57                 ps.setInt(1, id);
58                 ps.setString(2, request.getParameter("name"));
59
60                 ps.setInt(3, Integer.parseInt(request.getParameter("age")));
61                 ps.setString(4, request.getParameter("disease"));
62                 ps.setString(5, request.getParameter("treatment_status"));
63
64                 ps.executeUpdate();
65
66                 out.println("<p>Patient added successfully.</p>");
67             } else if (action.equals("Update")) {
68
69                 PreparedStatement ps = con.prepareStatement(
70                     "UPDATE patient SET name=?, age=?, disease=?, treatment_sta
71                     tus=? WHERE patient_id=?"
72                 );
73
74                 ps.setString(1, request.getParameter("name"));
75
76                 ps.setInt(2, Integer.parseInt(request.getParameter("age")));
77                 ps.setString(3, request.getParameter("disease"));
78                 ps.setString(4, request.getParameter("treatment_status"));
79                 ps.setInt(5, id);
80
81                 ps.executeUpdate();
82
83                 out.println("<p>Patient updated successfully.</p>");
84             } else if (action.equals("Delete")) {

```

```

85         PreparedStatement ps = con.prepareStatement(
86             "DELETE FROM patient WHERE patient_id=?"
87         );
88
89         ps.setInt(1, id);
90
91         ps.executeUpdate();
92
93         out.println("<p>Patient deleted successfully.</p>");
94
95     } else if (action.equals("View")) {
96
97         Statement stmt = con.createStatement();
98
99         ResultSet rs = stmt.executeQuery(
100             "SELECT * FROM patient"
101         );
102
103         out.println("<h3>Patient Records</h3>");
104
105         while (rs.next()) {
106             out.println(
107                 rs.getInt("patient_id") + " | " +
108                 rs.getString("name") + " | " +
109                 rs.getInt("age") + " | " +
110                 rs.getString("disease") + " | " +
111                 rs.getString("treatment_status") +
112                 "<br>"
113             );
114         }
115     }
116
117     con.close();
118
119 } catch (Exception e) {
120     out.println("<p>Error: " + e.getMessage() + "</p>");
121 }
122 }
123 %>
124
125 </body>
126 </html>

```

SQL CODE

```
1 use devansh5;
2
3 CREATE TABLE employee (
4     emp_id INT PRIMARY KEY,
5     name VARCHAR(50),
6     department VARCHAR(50),
7     salary DOUBLE
8 );
9
10 CREATE TABLE patient (
11     patient_id INT PRIMARY KEY,
12     name VARCHAR(50),
13     age INT,
14     disease VARCHAR(100),
15     treatment_status VARCHAR(50)
16 );
```

Exercise

Q1. Create a JSP-based application to add, update, delete, and view employee details.

Employee Management

Employee ID:

Name:

Department:

Salary:

Employee added successfully.

Employee Management

Employee ID:

Name:

Department:

Salary:

Employee Records

101 | Devansh | cse | 111111.0

Employee Management

Employee ID:

Name:

Department:

Salary:

Employee updated successfully.

Employee Management

Employee ID:

Name:

Department:

Salary:

Employee deleted successfully.

Q2. Create a JSP-based application to manage patient records using CRUD operations. Include fields such as patient ID, name, age, disease, and treatment status.

Patient Management

Patient ID:

Name:

Age:

Disease:

Treatment Status:

Patient added successfully.

Patient Management

Patient ID:

Name:

Age:

Disease:

Treatment Status:

Patient Records

101 | admin | 19 | cold | ongoing

Patient Management

Patient ID:

Name:

Age:

Disease:

Treatment Status:

Patient updated successfully.

Patient Management

Patient ID:

Name:

Age:

Disease:

Treatment Status:

Patient deleted successfully.

Conclusion

Thus, the experiment successfully demonstrated CRUD operations using JSP and a database. The four basic operations—Create, Read, Update, and Delete—were implemented for managing

database records. The experiment provided an understanding of how JSP, Servlets, and JDBC work together to develop dynamic database-driven web applications.